

Reviewer Pack — Witness-Complex Betti Sum Lower-Bounds Formula Depth

Entry 7887f040a80a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 05:24:32 UTC

Witness-Complex Betti Sum Lower-Bounds Formula Depth

Entry ID: 7887f040a80a **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 05:24:32 UTC

1. Conjecture

Field A (mathematical branch): Forman discrete Morse theory / mod-2 simplicial homology of the minimal-witness complex of a Boolean function **Field B** (complexity object): Karchmer-Wigderson De Morgan formula depth $D(f)$ for $f: \{0,1\}^n \rightarrow \{0,1\}$

Statement:

For every non-constant Boolean function $f: \{0,1\}^n \rightarrow \{0,1\}$, let $\Sigma f^{\min}$ be the simplicial complex on vertex set $[n]$ obtained as the down-closure of the inclusion-MINIMAL witness sets $W(x,y) = \{i: x_i \neq y_i\}$ ranging over pairs $(x,y) \in \{0,1\}^n \times \{0,1\}^n$, and let $\beta(f) = \sum_{k \geq 0} \dim \{F_2\} H_k(\Sigma f^{\min}; F_2)$ be its total mod-2 Betti number. Then the De Morgan (Boolean) formula depth satisfies $D(f) \geq \lceil \log_2(1 + \beta(f)) \rceil$. A single instance with decision-tree depth $DT(f) < \lceil \log_2(1 + \beta(f)) \rceil$ (which would imply $D(f) < \lceil \log_2(1 + \beta(f)) \rceil$) falsifies the conjecture.

Rationale (proposer's reasoning):

A KW protocol of depth d partitions the relation R_f into $\leq 2^d$ monochromatic rectangles, each labeled by a single witness coordinate i ; restricting $\Sigma f^{\min}$ to the vertices used by these labels yields a cover by $\leq 2^d$ stars (cones over a single vertex), and a Mayer-Vietoris / nerve argument forces $\beta(f) \leq 2^d - 1$ whenever such a cover exists. The minimal-witness complex thus encodes a 'shape obstruction' that is invisible to rank-, Fourier-, or sensitivity-based KW techniques. Discrete Morse / simplicial F_2 homology of witness complexes has appeared in <5 papers crossed with formula depth, giving a genuinely fresh combinatorial-topological angle on KW games.

Taxonomy category: KARCHMER_WIGDERSON (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c148f9bed5152348

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across all trials ($n \in \{3..8\}$, 30 random non-constant truth tables per n plus explicit families AND_n , OR_n , XOR_n , MAJ_n , $Th_{\lfloor n/2 \rfloor}^n$, $AND_a \circ OR_b$), compute $DT(f)$

and $\lceil \log_2(1+\beta(f)) \rceil$. Conjecture is SUPPORTED iff support fraction ≥ 0.95 AND no trial violates $DT(f) \geq \lceil \log_2(1+\beta(f)) \rceil$; FALSIFIED by any single counterexample.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.78	SAFE	SAFE
ALGEBRIZATION	SAFE	0.83	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Karchmer-Wigderson formula depth simplicial complex Betti - Boolean function witness complex homology lower bound formula - discrete Morse theory Boolean function complexity formula depth

Top relevant hits considered: - [<http://arxiv.org/abs/1802.03129v3>] Rank Selection and Depth Conditions for Balanced Simplicial Complexes - [<http://arxiv.org/abs/2107.05128v2>] Karchmer-Wigderson Games for Hazard-free Computation - [<http://arxiv.org/abs/2002.07444v1>] Multiparty Karchmer-Wigderson Games and Threshold Circuits - [<http://arxiv.org/abs/2003.09703v1>] Variance function of boolean additive convolution - [<http://arxiv.org/abs/2511.07739v3>] A Lower Bound for the Fourier Entropy of Boolean Functions on the Biased Hypercube - [<http://arxiv.org/abs/2110.02458v3>] Magnitude homology of graphs and discrete Morse theory on Asao-Izumihara complexes - [<http://arxiv.org/abs/2504.15729v2>] Strong discrete Morse theory - [<http://arxiv.org/abs/math/0406486v2>] The Morse Complex for a Morse Function on a Manifold with Corners

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from collections import defaultdict
from itertools import combinations, product

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_truth_table(n):
        return [tuple(random.randint(0, 1) for _ in range(n)) for _ in range(2**n)]

    def is_constant(f):
        return all(f[0] == f[i] for i in range(len(f)))

    def decision_tree_depth(f, n):
        memo = {}
```

```

def dp(sigma):
    if sigma in memo:
        return memo[sigma]
    if len(sigma) == 1:
        return 0
    max_depth = 0
    for i in range(n):
        new_sigma = list(sigma)
        new_sigma[i] = 0
        depth_0 = dp(tuple(new_sigma))
        new_sigma[i] = 1
        depth_1 = dp(tuple(new_sigma))
        if depth_0 == float('inf') or depth_1 == float('inf'):
            max_depth = float('inf')
            break
        max_depth = max(max_depth, 1 + max(depth_0, depth_1))
    memo[sigma] = max_depth
    return max_depth

sigma = tuple(random.choice([0, 1]) for _ in range(n))
return dp(sigma)

def compute_betti_number(f, n):
    vertices = list(range(n))
    simplices = [vertices]

    def down_closure(W):
        closure = set(W)
        new_elements = True
        while new_elements:
            new_elements = False
            for v in vertices:
                if v not in closure and any(w in closure for w in W):
                    closure.add(v)
                    new_elements = True
        return closure

    witness_sets = []
    for x, y in product(f, repeat=2):
        if f[x] == 1 and f[y] == 0:
            witness_set = [i for i in range(n) if x[i] != y[i]]
            witness_sets.append(witness_set)

    for W in witness_sets:
        simplices.extend(down_closure(W))

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        rank = 0
        for j in range(n):
            pivot_row = -1
            for i in range(rank, m):
                if A[i][j] == 1:
                    pivot_row = i
                    break
            if pivot_row != -1:

```

```

        A[pivot_row], A[rank] = A[rank], A[pivot_row]
        rank += 1
        for i in range(m):
            if i != rank - 1 and A[i][j] == 1:
                for k in range(n):
                    A[i][k] ^= A[rank - 1][k]

    return rank

betti_number = 0
for k in range(n + 1):
    boundary_matrix = []
    for i in range(len(simplices)):
        for j in range(i + 1, len(simplices)):
            if all(v in simplices[j] for v in simplices[i]):
                boundary_matrix.append([1 if v in simplices[j] and v not in simplices[i] else 0 for v in simplices[j]])
    betti_number += (-1) ** k * gaussian_elimination(boundary_matrix)

return betti_number

n_values = [3, 4, 5, 6, 7, 8]
total_trials = 30
support_count = 0
counterexample = ""

for n in n_values:
    truth_tables = generate_truth_table(n)
    explicit_families = [
        lambda x: all(x[i] == 1 for i in range(n)), # AND_n
        lambda x: any(x[i] == 1 for i in range(n)), # OR_n
        lambda x: x[n // 2] ^ any(x[i] == 1 for i in range(n) if i != n // 2), # XOR_n
        lambda x: sum(x[:n // 2]) >= n // 2, # MAJ_n
        lambda x: all(x[i] == 0 for i in range(n - n // 2)) or any(x[i] == 1 for i in range(n // 2)), # PARITY_n
        lambda x: (x[a] + x[b]) if a * b == n else None # AND_aOR_b
    ]

    for _ in range(total_trials):
        f = random.choice(truth_tables)
        if is_constant(f):
            continue

        dt_depth = decision_tree_depth(f, n)
        betti_number = compute_betti_number(f, n)
        required_depth = math.ceil(math.log2(1 + betti_number))

        if dt_depth < required_depth:
            counterexample = f"Decision tree depth {dt_depth} < required depth {required_depth}"
            return {
                "metric_name": "Decision Tree Depth",
                "metric_value": dt_depth,
                "instances_tested": 1,
                "conjecture_holds": False,
                "counterexample": counterexample
            }

        if dt_depth >= required_depth:
            support_count += 1

```



```
depth_0 = dp(tuple(new_sigma))
^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

[Previous line repeated 994 more times]

RecursionError: maximum recursion depth exceeded

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a RecursionError in the `decision_tree_depth` function before any trial data was produced, so neither the support nor falsification condition can be evaluated. | next: Rewrite `decision_tree_depth` iteratively (or raise `sys.setrecursionlimit` and add memoization on the restricted truth-table) and rerun the pre-registered trials for $n \in \{3..8\}$.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	281342
2	preregistration	claude_max	opus	0	0	6410
3	novelty	claude_max	opus	0	0	4975
4	novelty	claude_max	opus	0	0	7482
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	80069
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	197474
7	judge	claude_max	opus	0	0	4889

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 582640 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/7887f040a80a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/7887f040a80a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/7887f040a80a.tar.gz` (if generated)